

Employee Directory Report

Name: ZUO BOYU | **Matric Number:** A24CS4045 | **Course:** Cross Platform Application Development | **Chapter:** 8 – Connecting Vue to Backend

1. Working UI Screenshots

The application is a single-page Vue 3 frontend talking to an Express + MySQL backend. Figure 1 shows the laptop view with the summary cards, add/edit form, search and sort controls, paginated table, active/inactive badges, and Malaysian Ringgit salary formatting. Figure 2 shows the same UI re-flowed at tablet width: the form stacks above the table, summary cards wrap, and no horizontal scrolling is needed.

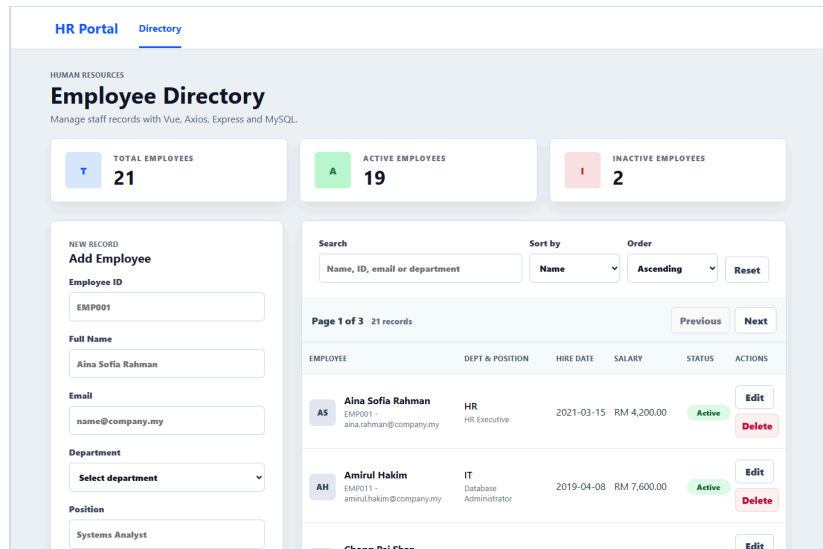


Figure 1. Laptop view (≥1024 px). Summary cards, paginated table with badges, RM-formatted salary, and edit/delete actions.

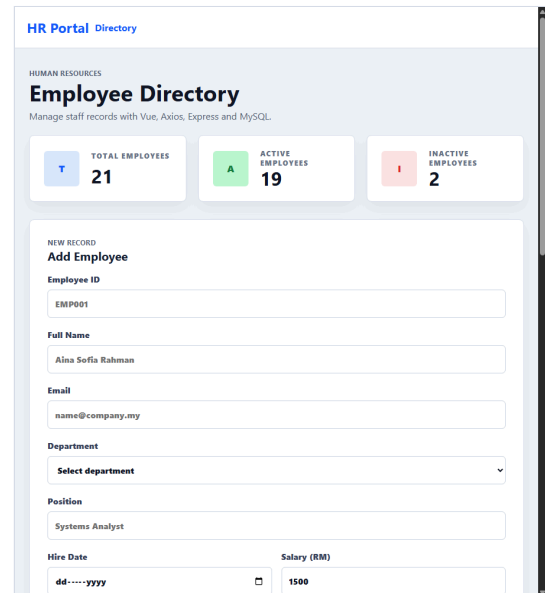


Figure 2. Tablet view (≥768 px). The CSS grid in `App.vue` collapses to a single column.

2. Chapter 8 Learning Outcomes – Code Evidence

Learning Outcome	How the project satisfies it	Where to look (file : line)
LO1. Connect Vue to a REST backend and manage the request/response lifecycle.	<code>App.vue</code> holds the <code>employees</code> array and a loading flag. <code>onMounted</code> triggers <code>loadEmployees()</code> , which awaits the API and updates state; errors surface via an <code>errorMessage</code> banner.	<code>src/App.vue:28</code> — <code>onMounted(loadEmployees)</code> <code>src/App.vue:30-53</code> — loading/error lifecycle <code>src/App.vue:148</code> — error banner
LO2. Configure and use Axios with interceptors and <code>async/await</code> .	A single <code>axios.create()</code> instance defines <code>baseUrl</code> , <code>timeout</code> , and default headers. A request interceptor logs every outgoing call; a response interceptor maps every error shape (server, network, timeout) to a single <code>{message, status, errors}</code> object. All call sites use <code>async/await</code> — no inline <code>axios.get(...)</code> in components.	<code>src/services/api.js:3</code> — single instance <code>src/services/api.js:12</code> — request interceptor <code>src/services/api.js:18</code> — response interceptor <code>src/services/api.js:35-52</code> — <code>async/await</code> calls
LO3. Build validated Vue forms using <code>v-model</code> and emit results.	<code>EmployeeForm.vue</code> uses <code>v-model.trim</code> on text fields and <code>v-model.number</code> on Salary. <code>validate()</code> enforces all seven brief rules — <code>^EMP[0-9]{3,5}\$</code> , <code>name ≥ 3</code> , email regex, department dropdown, position, hire date ≤ today, salary 1500–50000. Inline <code></code> messages render per	<code>src/components/EmployeeForm.vue:73-94</code> — rules <code>src/components/EmployeeForm.vue:135,177</code> — <code>.trim</code> / <code>.number</code> <code>src/components/EmployeeForm.vue:50</code> — <code>emit('save', ...)</code>

Learning Outcome	How the project satisfies it	Where to look (file : line)
	field, and the form short-circuits when any error exists. Validated data is sent to the parent via <code>emit('save', ...)</code> .	
LO4. Full CRUD against MySQL with prepared statements, server-side search and sort.	Express implements <code>GET/POST/PUT/DELETE /employees</code> . Search uses <code>LIKE</code> on four columns (name, empld, email, department). Sort is restricted to a whitelist map <code>{name, hireDate, salary, department}</code> — client input never reaches the SQL string. All values are passed as <code>? placeholders</code> to <code>pool.execute</code> ; pagination <code>LIMIT/OFFSET</code> are integer-clamped and parameterised. <code>mysql2/promise</code> provides the connection pool.	<code>server/index.js:12</code> — sort whitelist <code>server/index.js:26-73</code> — list/search/sort <code>server/index.js:89,120,154</code> — create/update/delete <code>server/db.js:4</code> — <code>mysql.createPool</code>

3. Architecture and Component Structure

- **Parent state:** `App.vue` owns the `employees` array, the active `filters`, pagination metadata, and the currently selected employee. Children never call the API directly.
- **Children & emits:** `EmployeeForm` emits `save / cancel`; `EmployeeList` emits `edit / delete / page-change`; `SearchSortControls` emits `change` with a debounced search and whitelisted sort keys.
- **Service layer:** `src/services/api.js` is the only file that imports `Axios`. Components import named functions (`fetchEmployees`, `createEmployee`, ...) and remain transport-agnostic.
- **Backend boundary:** validation runs twice — once in `EmployeeForm.validate()` for fast inline feedback, and once in `server/index.js:validateEmployee` as the authoritative gate before any SQL write. Duplicate `empId/email` are converted to friendly per-field errors via the MySQL `ER_DUP_ENTRY` handler.

4. Challenges and Resolutions

- **SQL injection on sort:** The brief warns against arbitrary column names in `ORDER BY`. Solved by mapping `req.query.sortBy` through a fixed whitelist object; unknown keys fall back to `name`. The direction is reduced to the literal string `'ASC'` or `'DESC'` before interpolation.
- **Heterogeneous Axios errors:** Server validation errors, 5xx responses, DNS failures, and timeouts each produce a different `Axios` error shape. The response interceptor normalises them into one `{message, status, errors}` object so the UI banner and per-field error display always work the same way.
- **IPv6 vs IPv4 on Windows/Laragon:** Node 18+ resolves `localhost` to `::1` while Laragon's MySQL listens on `127.0.0.1`. The pool and CORS allow-list both pin to `127.0.0.1` to avoid intermittent `ECONNREFUSED`.
- **Responsive table on tablet:** Six columns are hard to fit at 768 px. The workspace switches from a two-column grid to a single column and the table wraps in a horizontally-safe container, keeping the Actions column reachable without overflow.

5. Extensions Beyond the Brief

- Server-side pagination (`page`, `pageSize`) with clamped integers and a single SQL count query that also returns active/inactive totals in one round trip.
- Search extended to a fourth column (department) on top of the required three.
- Defensive server-side re-validation that mirrors the client rules, so the API stays safe even if it is called outside the Vue app.
- A summary-card row driven by `SUM(CASE WHEN active ...)` so the totals are always consistent with the filtered query.

6. Sources and References

The following official documentation was consulted during implementation; all code was written by the author.

- Vue 3 documentation — Composition API, `<script setup>`, props/emits: <https://vuejs.org/guide/>
- Vite configuration and dev server: <https://vitejs.dev/config/>
- Axios — instance configuration and interceptors: <https://axios-http.com/docs/interceptors>
- Express 5 routing and middleware: <https://expressjs.com/en/5x/api.html>
- mysql2 Promise pool and prepared statements: <https://github.com/sidorares/node-mysql2#using-prepared-statements>
- MDN — `Intl.NumberFormat` for MYR currency formatting: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/NumberFormat

Academic Integrity: This is my own work. AI assistants were used only for explanation and debugging; the architecture, validation rules, SQL, and component design were authored by me and I can explain every line on request.